ĐẠI HỌC BÁCH KHOA HÀ NỘI

BÁO CÁO PROJECT 1

Phân loại chữ số viết tay trong dataset MNIST bằng phương pháp SVM và CNN

PHẠM HOÀNG THÁI DƯƠNG
duong.pht235061@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: Cô Trần Thị Lan Anh

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 01/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

BÁO CÁO PROJECT 1

Phân loại chữ số viết tay trong dataset MNIST bằng phương pháp SVM và CNN

PHẠM HOÀNG THÁI DƯƠNG
duong.pht235061@sis.hust.edu.vn

Khoa học máy tính

Giảng viên hướng dẫn: Cô Trần Thị Lan Anh _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 01/2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Nhận dạng chữ số viết tay là bước đầu tiên của các hệ thống OCR (Optical character recognition – nhận diện ký tự quang học) trong ngân hàng, bưu điện. Tuy nhiên, sự khác biệt trong nét vẽ, độ nghiêng của nét chữ số, hay độ nhiễu trong hình ảnh khiến việc nhận diện không đơn giản.

1.2 Mục tiêu và định hướng giải pháp

Để giải quyết vấn đề này, ta đến với bài toán phân loại đa lớp. Trong học máy, đây là bài toán phân loại các mẫu dữ liệu vào một trong nhiều hơn hai lớp đối tượng, với nhiều hướng giải quyết, bao gồm các phương pháp thiên về học máy truyền thống, và các phương pháp thiên về học sâu. Với mục đích hiểu sâu hơn về cả học máy truyền thống và học sâu, em đã lựa chọn giải quyết bài toán bằng hai phương pháp: Support vector machine (SVM) và Convolutional neural network (CNN), lần lượt là các phương pháp được ứng dụng phổ biến trong học máy truyền thống và học sâu. Về bộ dữ liệu dành cho huấn luyện và kiểm tra, dataset MNIST, với 70000 ảnh chữ số viết tay đen trắng kích thước 24x24 pixel, là một lựa chọn phù hợp cho bài toán.

1.3 Bố cục đồ án

Trong chương 2, em sẽ trình bày các nền tảng lý thuyết cốt lõi liên quan đến bài toán phân loại đa lớp, và 2 phương pháp học máy SVM và CNN. Chương này sẽ làm rõ các khái niệm liên quan đến SVM, bao gồm bài toán tối ưu siêu phẳng chia cắt 2 bộ dữ liệu, soft margin support vector machine, và kernel support vector machine; và các khái niệm liên quan đến CNN, bao gồm mô hình neural network truyền thống, mô hình convolutional neural network, loss function, và backpropagation.

Tiếp nối các cơ sở lý thuyết, Chương 3 đi sâu vào quá trình thực nghiệm và đánh giá chi tiết hiệu năng của các mô hình trên bộ dữ liệu MNIST. Tại đây, quy trình tiền xử lý dữ liệu, thiết lập các tham số huấn luyện và kết quả thực nghiệm bao gồm độ chính xác, thời gian thực thi cùng ma trận nhầm lẫn sẽ được phân tích cụ thể để đưa ra cái nhìn khách quan về ưu điểm và hạn chế của từng phương pháp.

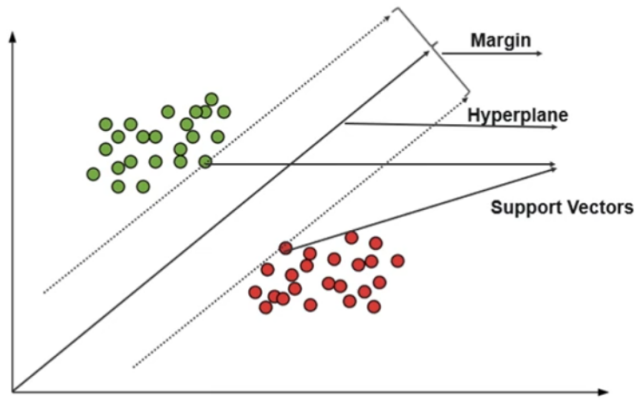
Cuối cùng, Chương 4 sẽ tổng kết lại những kết quả đạt được sau quá trình nghiên cứu, đồng thời đề xuất các hướng phát triển tiềm năng nhằm cải thiện độ chính xác và khả năng ứng dụng của mô hình trong thực tế.

CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT

2.1 Support Vector Machine (SVM)

2.1.1 Bài toán phân loại tuyến tính

Giả sử chúng ta có hai lớp (class) khác nhau được mô tả bởi các điểm trong không gian nhiều chiều. Trong trường hợp hai lớp này là phân tách tuyến tính (*linearly separable*), tồn tại một siêu phẳng (hyperplane) có thể phân chia chính xác các điểm dữ liệu thuộc về hai lớp đó.



Hình 2.1: Minh họa siêu phẳng phân chia trong không gian hai chiều.

Trên thực tế, có thể tồn tại nhiều mặt phân chia khác nhau. Thuật toán SVM tìm kiếm một siêu phẳng tối ưu sao cho khoảng cách từ điểm gần nhất của mỗi lớp đến mặt phân chia là bằng nhau và khoảng cách đó (gọi là lề - *margin*) đạt giá trị lớn nhất có thể.

2.1.2 Xây dựng bài toán tối ưu

Giả sử tập dữ liệu huấn luyện bao gồm các cặp dữ liệu $(\mathbf{x}_n, y_n)$ với $n = 1, 2, \ldots, N$. Trong đó:

- $\mathbf{x}_n \in \mathbb{R}^d$ là vector đầu vào d chiều, đại diện cho mẫu thứ n:

$$\mathbf{x}_n = \left(x_n^{(1)}, x_n^{(2)}, \ldots, x_n^{(d)}\right)^T \tag{2.1}$$

- $y_n \in \{1, -1\}$ là nhãn của dữ liệu.

Gọi siêu phẳng phân chia là $\mathbf{w}^T\mathbf{x} + b = 0$, trong đó vector trọng số $\mathbf{w}$ cũng là một vector d chiều:

$$\mathbf{w} = \left(w^{(1)}, w^{(2)}, \ldots, w^{(d)}\right)^T \tag{2.2}$$

Khi đó, khoảng cách hình học từ một mẫu dữ liệu $\mathbf{x}_n$ đến siêu phẳng được xác

định bởi công thức:

$$d(\mathbf{x}_n) = \frac{|\mathbf{w}^T\mathbf{x}_n + b|}{\|\mathbf{w}\|_2} \tag{2.3}$$

Trong đó, $\|\mathbf{w}\|_2$ là chuẩn bậc hai (L_2-norm) của vector trọng số, được tính bằng:

$$\|\mathbf{w}\|_2 = \sqrt{(w^{(1)})^2 + (w^{(2)})^2 + \cdots + (w^{(d)})^2} = \sqrt{\sum_{j=1}^{d}(w^{(j)})^2} \tag{2.4}$$

Tích vô hướng $\mathbf{w}^T\mathbf{x}_n$ trong công thức trên được khai triển dựa trên các thành phần đặc trưng như sau:

$$\mathbf{w}^T\mathbf{x}_n = \sum_{j=1}^{d} w^{(j)}x_n^{(j)} \tag{2.5}$$

Không mất tính tổng quát, chúng ta có thể giả sử rằng đối với các điểm dữ liệu thuộc lớp dương ($y_n = 1$), siêu phẳng phân chia thỏa mãn $\mathbf{w}^T\mathbf{x}_n + b > 0$. Ngược lại, với các điểm thuộc lớp âm ($y_n = -1$), ta có $\mathbf{w}^T\mathbf{x}_n + b < 0$. Khi đó, khoảng cách từ mỗi điểm $\mathbf{x}_n$ đến siêu phẳng có thể được viết gọn dưới dạng:

$$d_n = \frac{y_n(\mathbf{w}^T\mathbf{x}_n + b)}{\|\mathbf{w}\|_2} \tag{2.6}$$

Mục tiêu của thuật toán SVM là tìm bộ tham số $(\mathbf{w}, b)$ sao cho khoảng cách từ các điểm gần siêu phẳng nhất đến siêu phẳng đó là lớn nhất. Bài toán tối ưu được phát biểu như sau:

$$(\mathbf{w}, b) = \arg\max_{\mathbf{w},b}\left\{\min_{n=1\ldots N}\frac{y_n(\mathbf{w}^T\mathbf{x}_n + b)}{\|\mathbf{w}\|_2}\right\} \tag{2.7}$$

Do $\|\mathbf{w}\|_2$ là một hằng số đối với phép toán $\min_n$, ta có thể đưa nó ra ngoài:

$$(\mathbf{w}, b) = \arg\max_{\mathbf{w},b}\left\{\frac{1}{\|\mathbf{w}\|_2}\min_{n=1\ldots N} y_n(\mathbf{w}^T\mathbf{x}_n + b)\right\} \tag{2.8}$$

Giả thuyết về lề hàm số (Functional Margin)

Một quan sát quan trọng là siêu phẳng phân chia không thay đổi nếu chúng ta nhân cả $\mathbf{w}$ và b với cùng một hằng số dương $k \in \mathbb{R}^+$. Nói cách khác, phương trình $\mathbf{w}^T\mathbf{x} + b = 0$ và $(k\mathbf{w})^T\mathbf{x} + (kb) = 0$ biểu diễn cùng một siêu phẳng.

Vì lý do đó, chúng ta có thể tự do thay đổi độ lớn của $\mathbf{w}$ và b bằng cách chọn k sao cho đối với những điểm nằm gần siêu phẳng nhất, giá trị tuyệt đối của biểu

thức đại số bằng đúng 1:

$$\min_{n=1\ldots N} y_n(\mathbf{w}^T\mathbf{x}_n + b) = 1 \tag{2.9}$$

Khi đó, đối với mọi điểm dữ liệu trong tập huấn luyện, ta luôn có ràng buộc:

$$y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1, \quad \forall n = 1, 2, \ldots, N \tag{2.10}$$

Bài toán tối ưu chuẩn tắc

Nhờ giả thuyết trên, bài toán tối ưu ban đầu trở nên đơn giản hơn, chỉ còn phụ thuộc vào việc tối đa hóa giá trị $1/\|\mathbf{w}\|_2$:

$$(\mathbf{w}, b) = \arg\max_{\mathbf{w},b} \frac{1}{\|\mathbf{w}\|_2} \quad \text{thỏa mãn } y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1, \forall n \tag{2.11}$$

Để thuận tiện cho việc tính toán đạo hàm trong các bước sau, bài toán này thường được chuyển về dạng tương đương là cực tiểu hóa hàm bậc hai (dạng chuẩn tắc):

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|_2^2 \quad \text{thỏa mãn } 1 - y_n\left(\sum_{j=1}^{d} w^{(j)}x_n^{(j)} + b\right) \leq 0, \forall n \tag{2.12}$$

2.1.3 Bài toán đối ngẫu

Để giải bài toán tối ưu lồi với các ràng buộc bất đẳng thức, chúng ta sử dụng phương pháp nhân tử Lagrange để chuyển bài toán gốc (*Primal problem*) sang bài toán đối ngẫu (*Dual problem*).

a, Kiểm tra điều kiện Slater

Để đảm bảo tính chất đối ngẫu mạnh (*Strong Duality*), tức là giá trị tối ưu của bài toán gốc và bài toán đối ngẫu bằng nhau, điều kiện Slater cần được thỏa mãn.

Đối với bài toán SVM trong trường hợp phân tách tuyến tính, rõ ràng tồn tại bộ tham số $(\mathbf{w}_0, b_0)$ sao cho mọi điểm dữ liệu đều thỏa mãn ràng buộc:

$$1 - y_n(\mathbf{w}_0^T\mathbf{x}_n + b_0) \leq 0, \quad \forall n = 1, \ldots, N \tag{2.13}$$

Nếu ta chọn bộ tham số mới $(\mathbf{w}_1, b_1) = (2\mathbf{w}_0, 2b_0)$, khi đó:

$$1 - y_n(\mathbf{w}_1^T\mathbf{x}_n + b_1) = 1 - 2y_n(\mathbf{w}_0^T\mathbf{x}_n + b_0) \leq 1 - 2(1) = -1 < 0 \tag{2.14}$$

Vì tồn tại điểm $(\mathbf{w}_1, b_1)$ khiến các ràng buộc bất đẳng thức được thỏa mãn một cách

chặt chẽ (*strictly feasible*), điều kiện Slater được thỏa mãn. Điều này cho phép ta giải bài toán đối ngẫu thay cho bài toán gốc.

b, Hàm Lagrange

Hàm Lagrange của bài toán tối ưu SVM được xây dựng bằng cách kết hợp hàm mục tiêu và các ràng buộc kèm theo các nhân tử Lagrange $\lambda_n \geq 0$:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\lambda}) = \frac{1}{2}\|\mathbf{w}\|_2^2 + \sum_{n=1}^{N} \lambda_n \left(1 - y_n \left(\sum_{j=1}^{d} w^{(j)} x_n^{(j)} + b\right)\right) \tag{2.15}$$

Với $\lambda_n \geq 0, \forall n = 1, 2, \ldots, N$.

c, Hàm đối ngẫu Lagrange

Hàm đối ngẫu được xác định bằng cách cực tiểu hóa hàm $\mathcal{L}$ theo các biến $\mathbf{w}$ và b:

$$g(\boldsymbol{\lambda}) = \min_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\lambda}) \tag{2.16}$$

Để tìm giá trị cực tiểu, ta tính đạo hàm riêng của $\mathcal{L}$ theo $\mathbf{w}$ và b rồi cho bằng 0:

- Đạo hàm theo $\mathbf{w}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_{n=1}^{N} \lambda_n y_n \mathbf{x}_n = 0 \implies \mathbf{w} = \sum_{n=1}^{N} \lambda_n y_n \mathbf{x}_n \tag{2.17}$$

- Đạo hàm theo b:

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{n=1}^{N} \lambda_n y_n = 0 \tag{2.18}$$

Thay các biểu thức (2.17) và (2.18) vào phương trình hàm Lagrange (2.16), sau khi rút gọn, ta thu được hàm đối ngẫu Lagrange chỉ còn phụ thuộc vào $\boldsymbol{\lambda}$:

$$g(\boldsymbol{\lambda}) = \sum_{n=1}^{N} \lambda_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m y_n y_m \left(\sum_{j=1}^{d} x_n^{(j)} x_m^{(j)}\right) \tag{2.19}$$

Bài toán tối ưu SVM lúc này trở thành việc tìm $\boldsymbol{\lambda}$ để tối đa hóa hàm $g(\boldsymbol{\lambda})$ dưới các ràng buộc $\lambda_n \geq 0$ và $\sum \lambda_n y_n = 0$.

d, Giải bài toán đối ngẫu

Sau khi thu được hàm đối ngẫu $g(\boldsymbol{\lambda})$, mục tiêu của chúng ta là tìm bộ nhân tử Lagrange tối ưu:

$$\boldsymbol{\lambda}^* = \arg\max_{\boldsymbol{\lambda}} g(\boldsymbol{\lambda}) \tag{2.20}$$

thỏa mãn các ràng buộc:

- $\lambda_n \geq 0, \quad \forall n = 1, 2, \ldots, N$
- $\sum_{n=1}^{N} \lambda_n y_n = 0$

Đây là một bài toán tối ưu lồi dạng bậc hai (*Quadratic Programming - QP*). Trong thực tế triển khai, bài toán này có thể được giải quyết hiệu quả bằng các thư viện tối ưu hóa chuyên dụng như `CVXOPT` trong Python.

e, Điều kiện KKT (Karush-Kuhn-Tucker)

Vì bài toán SVM là bài toán tối ưu lồi và thỏa mãn điều kiện Slater (đối ngẫu mạnh), nghiệm tối ưu của bài toán gốc $(\mathbf{w}^*, b^*)$ và bài toán đối ngẫu $\boldsymbol{\lambda}^*$ phải thỏa mãn hệ điều kiện KKT đầy đủ sau đây:

1. **Tính khả thi của bài toán gốc (Primal Feasibility):**

$$1 - y_n \left(\sum_{j=1}^{d} w^{(j)} x_n^{(j)} + b \right) \leq 0, \quad \forall n = 1, \ldots, N \tag{2.21}$$

2. **Tính khả thi của bài toán đối ngẫu (Dual Feasibility):**

$$\lambda_n \geq 0, \quad \forall n = 1, \ldots, N \tag{2.22}$$

3. **Điều kiện bù trừ (Complementary Slackness):**

$$\lambda_n \left(1 - y_n \left(\sum_{j=1}^{d} w^{(j)} x_n^{(j)} + b \right) \right) = 0, \quad \forall n = 1, \ldots, N \tag{2.23}$$

4. **Điều kiện dừng (Stationarity):** Đạo hàm riêng của hàm Lagrange theo các biến gốc bằng 0 (đã trình bày ở phần trước):

$$\mathbf{w} = \sum_{n=1}^{N} \lambda_n y_n \mathbf{x}_n \quad \text{và} \quad \sum_{n=1}^{N} \lambda_n y_n = 0 \tag{2.24}$$

f, Vector hỗ trợ và Xác định tham số mô hình

Từ điều kiện bù trừ ở phương trình (2.23), chúng ta có thể rút ra những nhận định quan trọng về cấu trúc của mô hình:

- Với những điểm có $\lambda_n = 0$: Các điểm này không đóng góp vào việc xác định vector trọng số $\mathbf{w}$ và nằm ngoài vùng lề.

- Với những điểm có $\lambda_n > 0$: Để thỏa mãn phương trình (2.23), bắt buộc ta phải có $1 - y_n(\mathbf{w}^T\mathbf{x}_n + b) = 0$, hay $y_n(\mathbf{w}^T\mathbf{x}_n + b) = 1$.

Những điểm dữ liệu ứng với $\lambda_n > 0$ được gọi là các **Vector hỗ trợ (Support Vectors)**. Đây là những điểm nằm gần siêu phẳng phân chia nhất và trực tiếp quyết định vị trí của siêu phẳng đó.

Sau khi giải được $\boldsymbol{\lambda}$ bằng thư viện tối ưu, ta xác định các tham số của mô hình như sau:

1. **Tìm $\mathbf{w}$:** Thay các giá trị λ_n vừa tìm được vào công thức đạo hàm:

$$\mathbf{w}^* = \sum_{n \in \mathcal{S}} \lambda_n y_n \mathbf{x}_n \tag{2.25}$$

   với $\mathcal{S}$ là tập chỉ số của các Support Vectors.

2. **Tìm b:** Với bất kỳ một Support Vector $\mathbf{x}_k$ ($k \in \mathcal{S}$), ta có phương trình $y_k(\mathbf{w}^T\mathbf{x}_k + b) = 1$. Nhân cả hai vế với y_k (do $y_k^2 = 1$), ta thu được:

$$b^* = y_k - \mathbf{w}^{*T}\mathbf{x}_k = y_k - \sum_{j=1}^{d} w^{*(j)} x_k^{(j)} \tag{2.26}$$

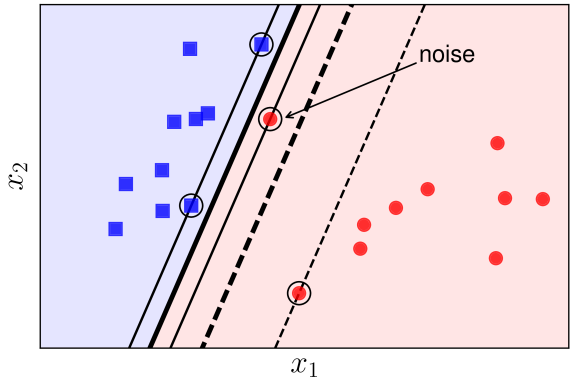
   Để kết quả ổn định hơn về mặt số học, giá trị b^* thường được tính bằng cách lấy trung bình cộng trên toàn bộ tập Support Vectors $\mathcal{S}$.

2.1.4 Soft Margin Support Vector Machine

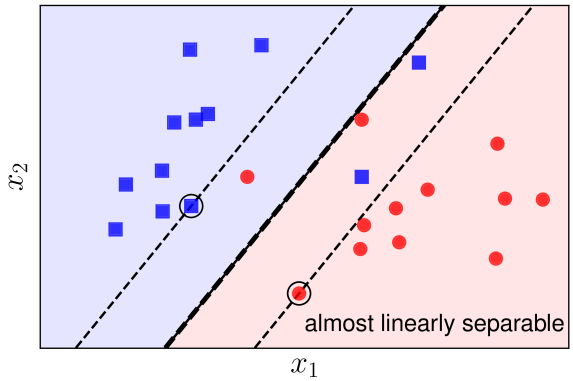
Trong thực tế, phương pháp Hard Margin SVM (lề cứng) gặp phải hai hạn chế lớn khiến mô hình hoạt động không hiệu quả hoặc thậm chí không thể tìm được nghiệm:

- **Trường hợp 1: Dữ liệu phân tách tuyến tính nhưng có nhiễu (noise).** Chỉ cần một điểm dữ liệu nhiễu nằm sát hoặc lấn sang phía lớp kia, lề của siêu phẳng sẽ bị thu hẹp đáng kể, dẫn đến việc mô hình trở nên quá nhạy cảm và kém ổn định.
- **Trường hợp 2: Dữ liệu không phân tách tuyến tính.** Khi các điểm dữ liệu của hai lớp đan xen lẫn nhau, không tồn tại bất kỳ siêu phẳng nào có thể phân chia tuyệt đối hai lớp, khiến bài toán Hard Margin không có nghiệm.

Để giải quyết vấn đề này, khái niệm **Soft Margin** (lề mềm) được đưa ra. Thay vì bắt buộc mọi điểm phải nằm đúng phía và ngoài vùng lề, chúng ta cho phép một số điểm "vi phạm" bằng cách nằm trong vùng lề hoặc thậm chí nằm sai phía của siêu phẳng. "Đường biên mềm" chính là siêu phẳng cho phép sự sai số này để đổi



Hình 2.2: Ảnh hưởng của điểm nhiễu đối với Hard Margin SVM.



Hình 2.3: Dữ liệu không thể phân tách tuyến tính.

lấy một hành lang (margin) rộng hơn và ổn định hơn.

a, Lập bài toán tối ưu

Để cụ thể hóa ý tưởng trên, với mỗi điểm dữ liệu $\mathbf{x}_n$, chúng ta giới thiệu một biến bù (*slack variable*) $\xi_n \geq 0$ (đọc là epsilon hoặc xi) đại diện cho "độ hy sinh" hay mức độ vi phạm của điểm đó:

- **Vùng an toàn:** Là khu vực nằm đúng phía của siêu phẳng và nằm ngoài hoặc vừa đúng trên đường biên lề ($y_n(\mathbf{w}^T\mathbf{x}_n + b) \geq 1$). Với các điểm nằm trong vùng này, $\xi_n = 0$.
- **Vùng vi phạm:** Nếu một điểm nằm trong hành lang lề hoặc sai phía siêu phẳng, ξ_n sẽ bằng khoảng cách từ điểm đó tới biên lề tương ứng của lớp nó. Khi đó, $\xi_n > 0$.

Thay vì ràng buộc cứng, chúng ta sử dụng ràng buộc mềm:

$$y_n \left(\sum_{j=1}^{d} w^{(j)} x_n^{(j)} + b \right) \geq 1 - \xi_n, \quad \xi_n \geq 0, \forall n = 1, \ldots, N \tag{2.27}$$

b, Hàm mục tiêu và tham số C

Bài toán tối ưu bây giờ không chỉ là tối đa hóa lề (tương đương cực tiểu hóa $\frac{1}{2}\|\mathbf{w}\|_2^2$) mà còn phải đồng thời cực tiểu hóa tổng các sai số ξ_n. Điều này dẫn đến hàm mục tiêu mới:

$$\min_{\mathbf{w}, b, \boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|_2^2 + C \sum_{n=1}^{N} \xi_n \tag{2.28}$$

Trong đó, $C > 0$ là tham số điều chuẩn (*regularization parameter*) đóng vai trò cân bằng giữa hai mục tiêu đối lập:

1. **Tối đa hóa lề:** Giúp mô hình đơn giản hơn và tổng quát hóa tốt hơn (tránh Overfitting).
2. **Cực tiểu hóa sai số:** Đảm bảo mô hình phân loại đúng càng nhiều điểm trên tập huấn luyện càng tốt.

Nếu chọn C rất lớn, mô hình sẽ ưu tiên việc phân loại đúng (tiến về Hard Margin). Nếu chọn C nhỏ, mô hình chấp nhận nhiều lỗi hơn để đổi lấy một đường biên phân cách rộng và mượt mà hơn.

c, Bài toán đối ngẫu Lagrange

Điều kiện Slater: Tương tự bài toán SVM truyền thống, ở đây điều kiện slater cũng được thỏa mãn

Hàm Lagrangian cho bài toán tối ưu trên được xác định bởi:

$$L(w, b, \epsilon, \lambda, \mu) = \frac{1}{2}\|w\|_2^2 + C \sum_{n=1}^{N} \epsilon_n + \sum_{n=1}^{N} \lambda_n (1 - \epsilon_n - y_n(w^T x_n + b)) - \sum_{n=1}^{N} \mu_n \epsilon_n \tag{2.29}$$

Trong đó $\lambda = (\lambda_1, \lambda_2, \ldots, \lambda_N)^T \succeq 0$ và $\mu = (\mu_1, \mu_2, \ldots, \mu_N)^T \succeq 0$ là các biến đối ngẫu Lagrange.

Để tìm cực trị, ta thiết lập đạo hàm riêng của L theo các biến gốc bằng 0:

$$\nabla_w L = 0 \iff w = \sum_{n=1}^{N} \lambda_n y_n x_n \tag{2.30}$$

$$\frac{\partial L}{\partial b} = 0 \iff \sum_{n=1}^{N} \lambda_n y_n = 0 \tag{2.31}$$

$$\frac{\partial L}{\partial \epsilon_n} = 0 \iff \lambda_n = C - \mu_n, \quad \forall n = 1, \ldots, N \tag{2.32}$$

Từ (2.32), kết hợp với điều kiện $\lambda_n \geq 0$ và $\mu_n \geq 0$, ta suy ra hệ quả quan trọng: $0 \leq \lambda_n \leq C$.

Thay các biểu thức (2.30), (2.31), và (2.32) vào hàm Lagrangian (2.29), ta thu được bài toán đối ngẫu:

$$\lambda = \arg\max_{\lambda} \left(\sum_{n=1}^{N} \lambda_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m y_n y_m x_n^T x_m \right) \tag{2.33}$$

với các điều kiện ràng buộc:

$$\sum_{n=1}^{N} \lambda_n y_n = 0 \quad \text{và} \quad 0 \leq \lambda_n \leq C, \forall n = 1, \ldots, N$$

Hệ điều kiện KKT (Karush-Kuhn-Tucker):

$$1 - \epsilon_n - y_n(w^T x_n + b) \leq 0 \tag{2.34}$$

$$-\epsilon_n \leq 0, \quad \lambda_n \geq 0, \quad \mu_n \geq 0 \tag{2.35}$$

$$\lambda_n(1 - \epsilon_n - y_n(w^T x_n + b)) = 0 \tag{2.36}$$

$$\mu_n \epsilon_n = 0 \tag{2.37}$$

—

Phân tích vị trí các điểm dữ liệu:

Dựa trên hệ điều kiện trên, ta xét các trường hợp của λ_n:

- **Trường hợp 1:** Nếu $\lambda_n = 0$. Từ (2.32) suy ra $\mu_n = C > 0$. Kết hợp với (2.37) ta có $\epsilon_n = 0$. Thay vào (2.34), ta thu được $y_n(w^T x_n + b) \geq 1$. $\implies$ Điểm x_n nằm trong **vùng an toàn** (đúng phía và ngoài lề).
- **Trường hợp 2:** Nếu $0 < \lambda_n < C$. Từ (2.32) suy ra $\mu_n = C - \lambda_n > 0$, dẫn đến

$\epsilon_n = 0$ theo (2.37). Vì $\lambda_n > 0$, từ (2.36) ta có $1 - 0 - y_n(w^T x_n + b) = 0 \iff y_n(w^T x_n + b) = 1$. $\implies$ Điểm x_n nằm **chính xác trên đường margin**.

- **Trường hợp 3:** Nếu $\lambda_n = C$. Từ (2.32) suy ra $\mu_n = 0$. Khi đó ϵ_n có thể nhận giá trị không âm bất kỳ. Từ (2.36), ta có $y_n(w^T x_n + b) = 1 - \epsilon_n$.
  - Nếu $0 < \epsilon_n \leq 1$: x_n nằm giữa đường biên và margin nhưng vẫn phân lớp đúng.
  - Nếu $\epsilon_n > 1$: x_n bị phân lớp sai.

  $\implies$ Điểm x_n là điểm vi phạm margin (nhiễu).

Những điểm ứng với $0 < \lambda_n \leq C$ được gọi là các **Support Vectors**. Ta có các tập hợp chỉ số sau:

- $S = \{n : 0 < \lambda_n \leq C\}$: Tập các Support Vectors.
- $M = \{n : 0 < \lambda_n < C\}$: Tập các điểm nằm chính xác trên margin.

Xác định tham số tối ưu:

$$w = \sum_{n \in S} \lambda_n y_n x_n \tag{2.38}$$

Với mỗi $n \in M$, ta có $y_n(w^T x_n + b) = 1$. Để ổn định số học, ta tính trung bình cộng b trên toàn bộ tập M:

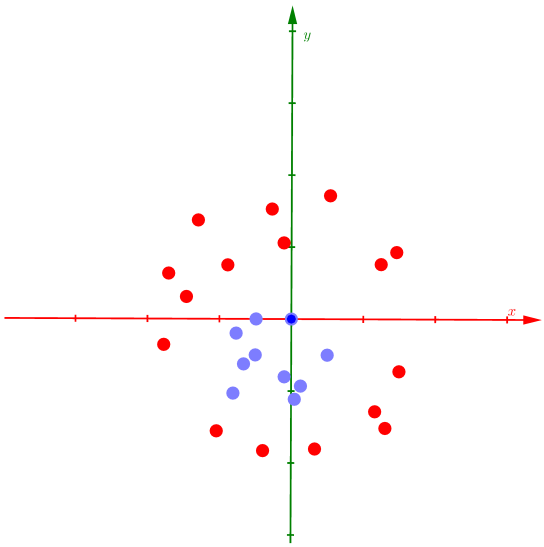
$$b = \frac{1}{|M|} \sum_{n \in M} (y_n - w^T x_n) \tag{2.39}$$

2.1.5 Kernel Support Vector Machine

Kernel Support Vector Machine (KSVM) là một mở rộng mạnh mẽ của phương pháp SVM truyền thống, được ứng dụng phổ biến trong các bài toán phân loại phức tạp như nhận dạng chữ số viết tay. Phương pháp này giải quyết vấn đề khi dữ liệu ban đầu không thể phân tách tuyến tính.

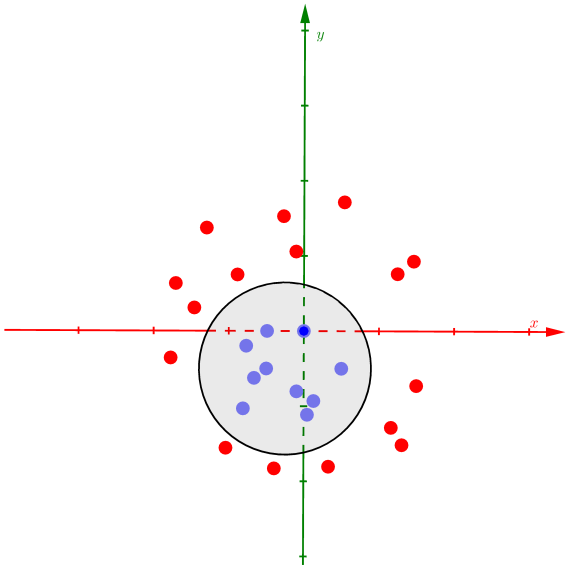
a, Ý tưởng về ánh xạ không gian (Feature Mapping)

Xét một tập dữ liệu trong không gian 2 chiều mà tại đó không tồn tại bất kỳ siêu phẳng (đường thẳng) nào có thể chia cắt các điểm dữ liệu thuộc hai lớp khác nhau (ví dụ: xanh và đỏ).



Hình 2.4: Dữ liệu phi tuyến tính trong không gian 2 chiều.

Tuy nhiên, qua quan sát, nếu các điểm thuộc một lớp tập trung gần gốc tọa độ hơn lớp còn lại, ta có thể vẽ một đường tròn bao quanh để phân tách chúng.

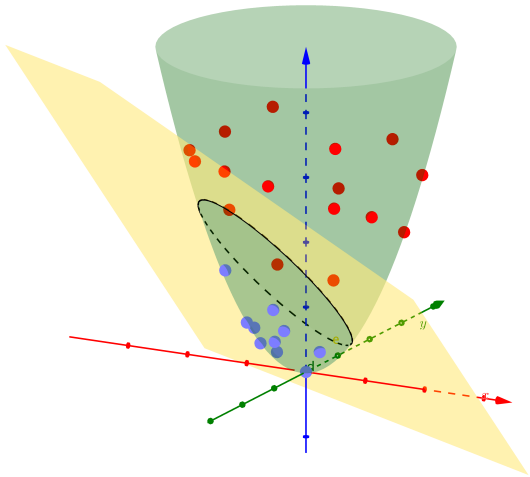


Hình 2.5: Đường tròn bao quanh bộ dữ liệu màu xanh

Thay vì cố gắng tìm một đường cong phức tạp trong không gian 2 chiều, người ta thực hiện ánh xạ dữ liệu sang không gian 3 chiều bằng cách thêm một thành phần z như sau:

$$z = (x^{(1)})^2 + (x^{(2)})^2 \tag{2.40}$$

Trong đó, $x^{(1)}$ và $x^{(2)}$ lần lượt là các thành phần tọa độ của vector dữ liệu $\mathbf{x}$.



Hình 2.6: Dữ liệu được ánh xạ sang không gian 3 chiều.

Khi đó, trong không gian mới, ta có thể dễ dàng xác định được một siêu phẳng chia cắt hoàn hảo hai tập dữ liệu. Kỹ thuật biến đổi dữ liệu từ không gian ban đầu sang không gian mới nhiều chiều hơn được gọi là **Kernel SVM**. Hàm biến đổi $\Phi(\mathbf{x})$ tổng quát được gọi là *feature mapping*, và hàm tính toán tích vô hướng trong không gian mới $K(\mathbf{x}, \mathbf{z}) = \Phi(\mathbf{x})^T\Phi(\mathbf{z})$ được gọi là **Kernel**.

b, Ví dụ về hàm Kernel

Một trong những hàm Kernel đơn giản và phổ biến nhất là **Polynomial Kernel** (Kernel đa thức). Với bậc $d = 2$ trong không gian 2 chiều, hàm Kernel có dạng:

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T\mathbf{z} + c)^2 \tag{2.41}$$

Hàm này tương ứng với việc ánh xạ dữ liệu vào không gian có các thành phần như $(x^{(1)})^2, (x^{(2)})^2, \sqrt{2}x^{(1)}x^{(2)}, \ldots$, giúp mô hình có thể tạo ra các đường biên phân loại dạng bậc hai (như hình tròn, ellipse, hay parabola) trong không gian gốc.

c, Bài toán tối ưu với Kernel Support Vector Machine

Để xây dựng mô hình Kernel SVM, trước hết ta hãy nhắc lại bài toán đối ngẫu trong Soft Margin SVM tại không gian đặc trưng ban đầu:

$$\lambda = \arg\max_{\lambda} \left(\sum_{n=1}^{N} \lambda_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m y_n y_m \mathbf{x}_n^T \mathbf{x}_m \right) \tag{2.42}$$

với điều kiện: $\sum_{n=1}^{N} \lambda_n y_n = 0$ và $0 \leq \lambda_n \leq C, \forall n = 1, \ldots, N$.

Khi áp dụng kỹ thuật ánh xạ dữ liệu vào không gian mới thông qua hàm $\Phi(\mathbf{x})$, mọi vector dữ liệu $\mathbf{x}$ sẽ được thay thế bằng $\Phi(\mathbf{x})$. Bài toán đối ngẫu lúc này trở

thành:

$$\lambda = \arg\max_{\lambda} \left(\sum_{n=1}^{N} \lambda_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m y_n y_m \Phi(\mathbf{x}_n)^T \Phi(\mathbf{x}_m) \right) \tag{2.43}$$

Việc xác định nhãn của một điểm dữ liệu mới $\mathbf{x}$ dựa trên dấu của biểu thức:

$$f(\mathbf{x}) = \mathbf{w}^T \Phi(\mathbf{x}) + b = \sum_{m \in S} \lambda_m y_m \Phi(\mathbf{x}_m)^T \Phi(\mathbf{x}) + b \tag{2.44}$$

Trong đó S là tập hợp các chỉ số của các Support Vectors, và tham số b được tính dựa trên một điểm $n \in M$ (điểm nằm chính xác trên margin):

$$b = y_n - \sum_{m \in S} \lambda_m y_m \Phi(\mathbf{x}_m)^T \Phi(\mathbf{x}_n) \tag{2.45}$$

Kỹ thuật Kernel (The Kernel Trick): Trong thực tế, việc tính toán trực tiếp $\Phi(\mathbf{x})$ có thể gặp hai trở ngại lớn:

1. **Chi phí tính toán:** Hàm Φ có thể ánh xạ dữ liệu vào không gian có số chiều rất lớn (thậm chí là vô hạn như Kernel RBF).
2. **Bộ nhớ:** Việc lưu trữ các vector $\Phi(\mathbf{x})$ trong không gian cao chiều cực kỳ tốn kém bộ nhớ.

Tuy nhiên, quan sát phương trình (2.44) và (2.45), ta nhận thấy dữ liệu chỉ xuất hiện dưới dạng tích vô hướng của hai vector trong không gian mới: $\Phi(\mathbf{x})^T \Phi(\mathbf{z})$. Điều này dẫn đến định nghĩa của hàm Kernel:

$$K(\mathbf{x}, \mathbf{z}) = \Phi(\mathbf{x})^T \Phi(\mathbf{z}) \tag{2.46}$$

Hàm Kernel cho phép ta tính trực tiếp giá trị tích vô hướng trong không gian cao chiều mà không cần thực hiện bước ánh xạ trung gian hay biết cụ thể dạng tường minh của $\Phi(\mathbf{x})$.

Ta phát biểu lại bài toán tối ưu với Kernel như sau:

$$\lambda = \arg\max_{\lambda} \left(\sum_{n=1}^{N} \lambda_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \lambda_n \lambda_m y_n y_m K(\mathbf{x}_n, \mathbf{x}_m) \right) \tag{2.47}$$

với các điều kiện ràng buộc giữ nguyên. Công thức xác định nhãn cho dữ liệu mới

cũng được viết lại gọn nhẹ như sau:

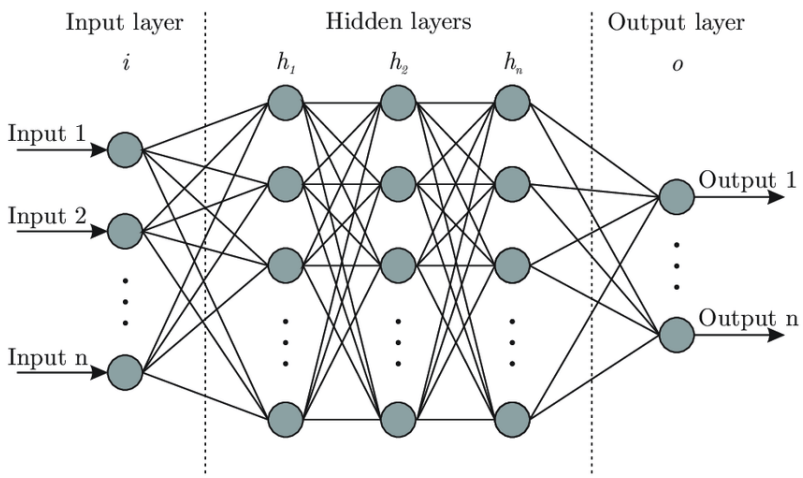
$$y_{new} = \text{sgn}\left(\sum_{m \in S} \lambda_m y_m K(\mathbf{x}_m, \mathbf{x}) + b\right) \tag{2.48}$$

với $b = \frac{1}{|M|} \sum_{n \in M} \left(y_n - \sum_{m \in S} \lambda_m y_m K(\mathbf{x}_m, \mathbf{x}_n)\right)$.

2.2 Convolutional Neural Network (CNN)

2.2.1 Mạng nơ-ron truyền thống (Fully Connected Network)

Trước khi đi sâu vào kiến trúc CNN, ta hãy xem xét mô hình mạng nơ-ron truyền thống, nơi mỗi nơ-ron ở tầng này kết nối với toàn bộ nơ-ron ở tầng kế tiếp (Fully Connected Layer).



Hình 2.7: Mô hình mạng nơ-ron truyền thống với các tầng liên kết đầy đủ.

Tuy nhiên, đối với dữ liệu hình ảnh, mạng liên kết đầy đủ bộc lộ những hạn chế lớn:

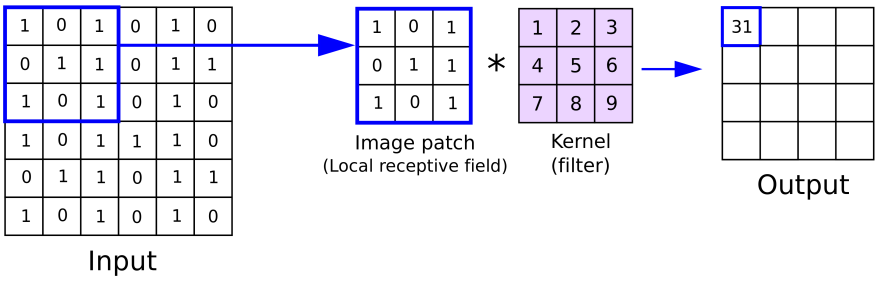
- **Sự bùng nổ tham số:** Xét một ảnh đầu vào kích thước 64×64 pixel (tương đương 4096 điểm ảnh). Nếu tầng ẩn đầu tiên chỉ có 1000 nơ-ron, số lượng tham số cần huấn luyện đã lên tới $4096 \times 1000 \approx 4.1$ triệu tham số. Với ảnh màu hoặc mạng nhiều tầng, con số này sẽ cực kỳ lớn, dẫn đến hiện tượng quá khớp (overfitting) và chi phí tính toán khổng lồ.
- **Mất thông tin không gian:** Mạng truyền thống yêu cầu "phẳng hóa" (flatten) ảnh thành một vector dài, điều này làm mất đi mối tương quan giữa các pixel lân cận — vốn là thông tin quan trọng nhất để nhận diện hình khối và kết cấu.

2.2.2 Mạng nơ-ron tích chập (CNN)

CNN ra đời để giải quyết các nhược điểm trên bằng cách sử dụng các kết nối cục bộ và chia sẻ trọng số.

a, Kernel và Bias

Trong bài toán MNIST, ảnh đầu vào thường được chuẩn hóa về dạng ma trận 64×64. Thay vì kết nối mọi pixel, CNN sử dụng các bộ lọc (Kernel).



Hình 2.8: Quá trình trượt Kernel trên ảnh đầu vào để trích xuất đặc trưng.

- **Kernel:** Là một ma trận trọng số nhỏ (thường là 3×3 hoặc 5×5). Kernel sẽ trượt tuần tự trên ảnh đầu vào, thực hiện phép nhân chập (convolution) với từng vùng dữ liệu con.
- **Tính toán:** Tại mỗi vị trí, kết quả là tổng tích các phần tử tương ứng, sau đó cộng thêm một giá trị độ chệch (*bias*):

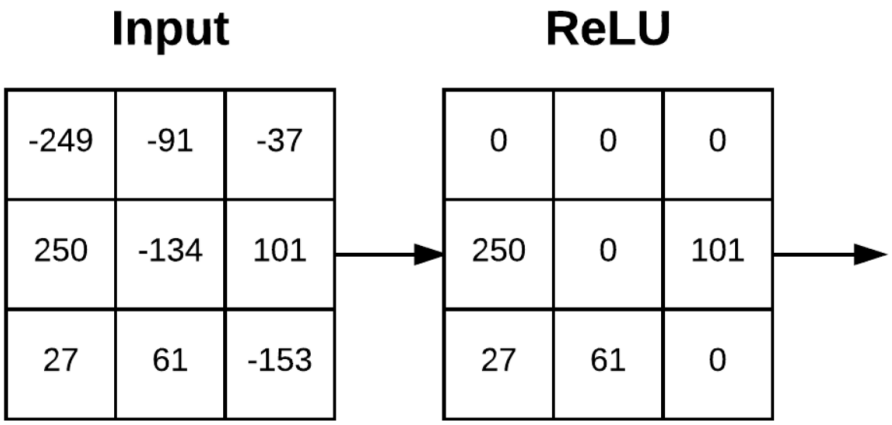
$$O_{i,j} = \sum_m \sum_n (I_{i+m,j+n} \cdot K_{m,n}) + b \tag{2.49}$$

Trong đó I là ma trận đầu vào, K là Kernel và b là bias.

Kết quả của quá trình này tạo ra một ma trận mới gọi là **Feature Map**, đại diện cho các đặc trưng như cạnh, góc hoặc vân bề mặt của ảnh.

b, Hàm kích hoạt (Activation Function)

Sau phép tính tích chập, các giá trị đầu ra thường được đưa qua một hàm kích hoạt phi tuyến để mô hình có thể học được các quy luật phức tạp. Trong các kiến trúc CNN hiện đại, hàm **ReLU (Rectified Linear Unit)** là lựa chọn phổ biến nhất.



Hình 2.9: Đồ thị hàm kích hoạt ReLU.

Công thức của hàm ReLU rất đơn giản:

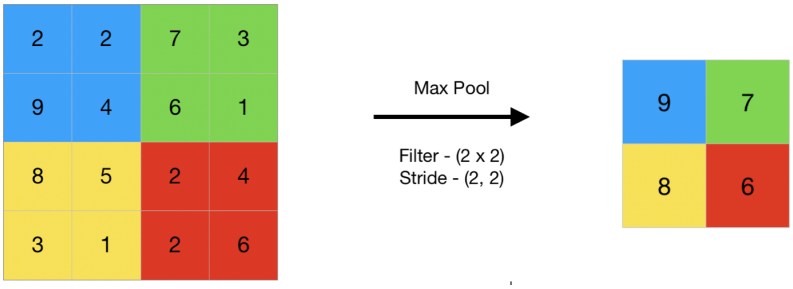
$$f(x) = \max(0, x) \tag{2.50}$$

Việc sử dụng ReLU giúp loại bỏ các giá trị âm (vốn không mang lại ý nghĩa đóng góp cho đặc trưng được trích xuất), đồng thời giúp mạng hội tụ nhanh hơn và giảm thiểu hiện tượng triệt tiêu đạo hàm (vanishing gradient) so với các hàm như Sigmoid hay Tanh.

c, Lớp lấy mẫu cực đại (Max Pooling)

Sau tầng tích chập, ta thường sử dụng tầng Pooling để giảm kích thước không gian của biểu diễn (giảm số lượng tham số) nhưng vẫn giữ lại được những đặc trưng quan trọng nhất. Phổ biến nhất là **Max Pooling** với kích thước cửa sổ 2×2 và bước nhảy (*stride*) bằng 2.

- **Khái niệm Stride:** Stride là độ dời của cửa sổ trượt sau mỗi lần tính toán. Ví dụ, với $stride = 2$, sau khi tính toán tại một vùng, cửa sổ sẽ nhảy cách sang bên phải (hoặc xuống dưới) 2 pixel thay vì 1. Điều này giúp giảm đáng kể kích thước của ma trận đầu ra.
- **Cơ chế hoạt động:** Khi cửa sổ trượt qua ma trận đặc trưng, nó chỉ trích xuất giá trị lớn nhất trong vùng đó.



Hình 2.10: Minh họa cơ chế Max Pooling giúp giảm chiều dữ liệu.

Việc chỉ giữ lại giá trị cực đại giúp mạng nơ-ron có khả năng "bất biến đối với các dịch chuyển nhỏ" (translation invariance), đồng thời giúp mô hình tập trung vào những đặc trưng mạnh nhất đã tìm được.

d, Lớp kết nối đầy đủ (Fully Connected Layer - FC)

Sau khi dữ liệu đã đi qua nhiều lớp tích chập và pooling để trích xuất các đặc trưng cấp cao, ta cần một cơ chế để tổng hợp các thông tin này lại phục vụ cho việc phân loại.

- **Quá trình làm phẳng (Flattening):** Dữ liệu từ dạng ma trận 2D hoặc tensor 3D sẽ được trải phẳng thành một vector 1D duy nhất.
- **Vai trò:** Tầng FC hoạt động tương tự như một mạng nơ-ron truyền thống. Tại đây, mọi nơ-ron của tầng trước được kết nối với mọi nơ-ron của tầng sau. Nó đóng vai trò như một bộ phân loại cuối cùng, kết hợp các đặc trưng cục bộ đã được học để đưa ra quyết định tổng thể về đối tượng trong ảnh.

e, Hàm mục tiêu và Phân loại (Loss Function)

Giả sử lớp đầu ra cuối cùng của mạng là vector $\mathbf{z} = (z_1, z_2, \ldots, z_n)$, trong đó n là số lượng lớp cần phân loại (với MNIST là 10 lớp từ 0 đến 9). Các giá trị này (logits) có thể là số thực bất kỳ.

Để chuyển đổi các giá trị này thành xác suất có ý nghĩa (nằm trong đoạn $[0, 1]$ và tổng bằng 1), ta sử dụng hàm **Softmax**:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}} \tag{2.51}$$

Hàm mất mát Cross-Entropy (Cross-Entropy Loss): Trong quá trình huấn luyện, ta cần một thước đo để biết dự đoán của mạng ($\hat{y}$) đang cách xa nhãn thực tế (y) bao nhiêu. Hàm mất mát Cross-Entropy thường được ưu tiên sử dụng cho bài toán phân loại vì nó giúp quá trình tối ưu diễn ra nhanh hơn so với hàm lỗi bình phương trung bình (MSE).

$$L = -\sum_{i=1}^{n} y_i \log(\hat{y}_i) \tag{2.52}$$

[Image of Softmax and Cross-Entropy loss visualization]

Ý nghĩa của hàm Loss: Giả sử một hình ảnh có nhãn thực tế là lớp i (tức $y_i = 1$ và các $y_j = 0$ với $j \neq i$).

- Nếu mạng dự đoán chính xác ($\hat{y}_i$ tiến gần đến 1), giá trị $-\log(\hat{y}_i)$ sẽ rất nhỏ, tiệm cận về 0.
- Ngược lại, nếu mạng dự đoán sai ($\hat{y}_i$ tiến gần về 0), giá trị $-\log(\hat{y}_i)$ sẽ tiến tới dương vô cùng.

Mục tiêu của việc huấn luyện là tối thiểu hóa hàm L. Khi L nhỏ nhất, xác suất dự đoán tại lớp đúng sẽ đạt cực đại, đồng nghĩa với việc mạng nơ-ron đã học được cách phân loại chính xác dữ liệu đầu vào.

f, Lan truyền ngược (Backpropagation)

Mục tiêu tối thượng của quá trình huấn luyện mạng nơ-ron là tìm ra bộ tham số tối ưu cho các bộ lọc (Kernel K) và độ chệch (Bias b) tại mọi tầng, sao cho hàm mất mát L đạt giá trị nhỏ nhất.

Quá trình khởi tạo và Lan truyền tiến: Ban đầu, các tham số trong mạng (trọng số của các Kernel và các Bias) được khởi tạo một cách ngẫu nhiên. Khi một dữ liệu đầu vào $\mathbf{x}$ đi qua mạng, nó trải qua một chuỗi các phép biến đổi phi tuyến thông qua n tầng ($f_1, f_2, \ldots, f_n$):

$$\hat{y} = f_n(f_{n-1}(\ldots f_1(\mathbf{x}; K, b) \ldots)) \tag{2.53}$$

Sau bước này, ta tính được giá trị mất mát L dựa trên sự sai khác giữa $\hat{y}$ và nhãn thực tế y.

Giải thuật Lan truyền ngược và Quy tắc chuỗi (Chain Rule): Để cập nhật các tham số, ta cần biết mức độ ảnh hưởng của từng tham số đối với lỗi tổng thể. Điều này được thực hiện bằng cách tính đạo hàm riêng của hàm mất mát L theo từng tham số K và b thông qua quy tắc chuỗi:

- **Đối với Kernel (K):**

$$\frac{\partial L}{\partial K} = \frac{\partial L}{\partial f_n} \cdot \frac{\partial f_n}{\partial f_{n-1}} \cdot \cdots \cdot \frac{\partial f_1}{\partial K} \tag{2.54}$$

- **Đối với Bias (b):**

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial f_n} \cdot \frac{\partial f_n}{\partial f_{n-1}} \cdot \dots \cdot \frac{\partial f_1}{\partial b} \tag{2.55}$$

Ý nghĩa của các biểu thức này là ta đang lan truyền sai số từ tầng cuối cùng ngược trở lại tầng đầu tiên. Mỗi tầng sẽ nhận một phần "trách nhiệm" cho lỗi dự đoán, từ đó xác định hướng cần thay đổi các tham số.

Cập nhật tham số (Gradient Descent): Sau khi tính được các đạo hàm (gradient), các tham số sẽ được cập nhật theo hướng làm giảm hàm mất mát:

$$K_{new} = K_{old} - \eta \cdot \frac{\partial L}{\partial K} \tag{2.56}$$

$$b_{new} = b_{old} - \eta \cdot \frac{\partial L}{\partial b} \tag{2.57}$$

Trong đó η (Learning Rate) là tốc độ học, điều chỉnh mức độ thay đổi của tham số sau mỗi bước lặp. Quá trình Lan truyền tiến và Lan truyền ngược này được lặp đi lặp lại hàng nghìn lần trên toàn bộ tập dữ liệu (Epochs) cho đến khi mô hình hội tụ và đạt được độ chính xác mong muốn.

CHƯƠNG 3. ÁP DỤNG VÀO BÀI TOÁN THỰC TẾ

3.1 Thực nghiệm với mô hình SVM

Trong phần này, chúng ta sẽ đi sâu vào quá trình triển khai mô hình SVM để phân loại bộ dữ liệu MNIST, từ khâu xử lý dữ liệu thô đến việc thiết lập cấu hình huấn luyện tối ưu.

3.1.1 Tiền xử lý dữ liệu (Preprocessing)

Dữ liệu hình ảnh từ bộ MNIST cần được đưa về định dạng phù hợp trước khi đưa vào mô hình SVM. Quy trình thực hiện gồm các bước chính sau:

- **Chuẩn hóa (Normalization):** Các giá trị pixel trong ảnh gốc nằm trong khoảng $[0, 255]$. Chúng ta thực hiện chuẩn hóa về đoạn $[0.0, 1.0]$ bằng công thức:

$$X_{norm} = \frac{X_{raw}}{255.0} \tag{3.1}$$

  Việc chuẩn hóa giúp các giá trị đặc trưng có cùng quy mô, hỗ trợ hàm Kernel RBF tính toán khoảng cách chính xác hơn và tăng tốc độ hội tụ của thuật toán.

- **Làm phẳng dữ liệu (Flattening):** SVM yêu cầu đầu vào là các vector đặc trưng một chiều. Do đó, mỗi ma trận ảnh kích thước 28×28 được trải phẳng thành một vector có độ dài 784:

$$X \in \mathbb{R}^{28\times 28} \xrightarrow{\text{Flatten}} X' \in \mathbb{R}^{784} \tag{3.2}$$

- **Quản lý nhãn:** Nhãn dữ liệu được giữ nguyên dưới dạng số nguyên từ 0 đến 9 để phục vụ cho chiến lược phân loại đa lớp.

Trong quá trình thực nghiệm, do SVM có độ phức tạp tính toán khá cao ($O(N^2 \cdot d)$ đến $O(N^3 \cdot d)$), mô hình được thiết kế để có thể huấn luyện trên một tập con (*subset*) dữ liệu nếu cần thiết, nhằm tối ưu hóa thời gian thử nghiệm trước khi áp dụng trên toàn bộ các mẫu.

3.1.2 SVM trong phân loại đa lớp

Về bản chất toán học, thuật toán SVM được thiết kế để giải quyết bài toán phân loại nhị phân (Binary Classification) — tức là chỉ phân biệt giữa hai lớp dữ liệu dựa trên một siêu phẳng tối ưu. Tuy nhiên, bộ dữ liệu MNIST có 10 lớp chữ số từ 0 đến 9. Để giải quyết vấn đề này, thư viện `sklearn.svm.SVC` áp dụng chiến lược **One-vs-One (OvO)**.

Cơ chế hoạt động của chiến lược One-vs-One: Trong chiến lược OvO, thay vì xây dựng một bộ phân loại duy nhất cho 10 lớp, hệ thống sẽ xây dựng các bộ phân loại nhị phân cho từng cặp lớp có thể có.

Số lượng bộ phân loại nhị phân cần thiết được tính theo công thức tổ hợp:

$$N_{classifiers} = \frac{k(k-1)}{2} \tag{3.3}$$

Trong đó k là số lượng lớp. Với bài toán MNIST ($k = 10$), số lượng bộ phân loại được tạo ra là:

$$N_{classifiers} = \frac{10(10-1)}{2} = 45 \text{ bộ phân loại nhị phân} \tag{3.4}$$

Mỗi bộ phân loại nhị phân (ví dụ: bộ phân loại "Số 1 vs Số 2", "Số 4 vs Số 9",...) sẽ chỉ tập trung vào việc học cách phân biệt giữa hai chữ số cụ thể đó dựa trên tập dữ liệu tương ứng.

Cách ra quyết định của AI (Cơ chế bỏ phiếu - Voting): Khi một hình ảnh chữ số mới (vector X') được đưa vào hệ thống để dự đoán, quá trình ra quyết định diễn ra như sau:

1. **Dự đoán song song:** Hình ảnh này sẽ được đưa qua toàn bộ 45 bộ phân loại nhị phân đã huấn luyện.
2. **Tích lũy phiếu bầu:** Mỗi bộ phân loại sẽ đưa ra một "phiếu bầu" cho lớp mà nó cho là đúng nhất.
   - Ví dụ: Bộ phân loại "3 vs 8" nhận định ảnh là số 3, thì lớp "3" sẽ được cộng thêm 1 điểm.
3. **Quyết định cuối cùng:** Lớp nào nhận được nhiều phiếu bầu nhất từ tổng số 45 dự đoán sẽ được AI chọn làm nhãn dự đoán cuối cùng cho hình ảnh đó.

3.1.3 Triển khai (Implement)

a, Sơ đồ cấu trúc thư mục (Project Structure)

Dự án phân loại chữ số MNIST sử dụng SVM được tổ chức theo cấu trúc thư mục sau để đảm bảo tính mô đun và dễ bảo trì:

- **main.py:** File chính để chạy chương trình, chứa các hàm `run_svm_project()` và `run_cnn_project()` để thực thi các mô hình.
- **Data_preparation/:** Thư mục chứa các module xử lý dữ liệu.
  - `__init__.py`: File khởi tạo package.

  - `data_loader.py`: Chứa các hàm tải và chuẩn bị dữ liệu cho SVM và CNN, bao gồm chuẩn hóa và làm phẳng dữ liệu.

- **Models/:** Thư mục chứa các module định nghĩa mô hình.
  - `__init__.py`: File khởi tạo package.
  - `svm_model.py`: Chứa các hàm huấn luyện, lưu, tải và đánh giá mô hình SVM.
  - `cnn_model.py`: Chứa các hàm xây dựng, huấn luyện và đánh giá mô hình CNN.

- **Results/:** Thư mục lưu trữ kết quả và mô hình đã huấn luyện.
  - `svm_model.joblib`: Mô hình SVM đã được lưu.
  - `Result SVM.txt`: Kết quả đánh giá của mô hình SVM.

b, Quá trình huấn luyện AI

Quá trình huấn luyện mô hình SVM được thực hiện theo các bước chi tiết sau:

1. **Tải và tiền xử lý dữ liệu:**
   Sử dụng hàm `load_and_prepare_for_svm()` từ module `Data_preparation/da`
   Dữ liệu MNIST được tải từ thư viện TensorFlow/Keras, bao gồm 60.000 mẫu huấn luyện và 10.000 mẫu kiểm tra. Các bước tiền xử lý bao gồm:
   - Chuẩn hóa giá trị pixel từ khoảng [0, 255] về [0.0, 1.0] bằng cách chia cho 255.0.
   - Làm phẳng (flatten) mỗi ảnh 28×28 thành vector 784 chiều để phù hợp với đầu vào của SVM.
   - Giữ nguyên nhãn dưới dạng số nguyên từ 0 đến 9.
2. **Huấn luyện mô hình:**
   Sử dụng hàm `train_svm()` từ module `Models/svm_model.py`. Mô hình SVM được cấu hình với:
   - Kernel RBF (Radial Basis Function) - phù hợp cho dữ liệu không tuyến tính.
   - Tham số C = 1.0 (điều chỉnh độ phức tạp của mô hình).
   - Tham số gamma = 'scale' (tự động điều chỉnh dựa trên dữ liệu).
   - Random state = 42 để đảm bảo tính tái lập.
3. **Lưu trữ mô hình:**
   Mô hình đã huấn luyện được lưu vào file `Results/svm_model.joblib`

bằng thư viện joblib để có thể tái sử dụng mà không cần huấn luyện lại.

c, Giải thích các hàm quan trọng (Code Snippets)

Trong phần triển khai SVM, các hàm quan trọng được định nghĩa trong file `Models/svm_model.py` và được sử dụng trong `main.py`. Dưới đây là giải thích chi tiết các hàm chính:

Hàm `train_svm(X_train, y_train, max_samples=None)`: Hàm này chịu trách nhiệm huấn luyện mô hình SVM với kernel RBF. Đầu vào là dữ liệu huấn luyện đã được làm phẳng và nhãn tương ứng. Tham số `max_samples` cho phép giới hạn số lượng mẫu để thử nghiệm nhanh. Hàm sử dụng `SVC` từ thư viện scikit-learn với các tham số mặc định phù hợp cho dữ liệu ảnh.

Hàm `save_model(model, filename)` và `load_model(filename)`: Hai hàm này sử dụng thư viện `joblib` để lưu và tải mô hình SVM đã huấn luyện. Việc lưu trữ mô hình giúp tránh phải huấn luyện lại từ đầu, tiết kiệm thời gian trong các lần chạy sau.

Hàm `evaluate_svm(model, X_test, y_test)`: Hàm đánh giá hiệu suất của mô hình trên tập kiểm tra. Nó tính toán độ chính xác, hiển thị ma trận nhầm lẫn và báo cáo phân loại chi tiết cho từng lớp. Điều này giúp phân tích các lỗi phổ biến trong quá trình phân loại.

Trong `main.py`, hàm `run_svm_project()` tích hợp tất cả các bước: tải dữ liệu, huấn luyện, lưu mô hình và đánh giá. Quá trình này đảm bảo luồng xử lý hoàn chỉnh từ dữ liệu thô đến kết quả cuối cùng.

3.2 Thực nghiệm với mô hình CNN

Trong phần này, chúng ta sẽ khám phá việc áp dụng Mạng Nơ-ron Tích chập (CNN) để phân loại bộ dữ liệu MNIST. CNN tận dụng cấu trúc không gian của dữ liệu hình ảnh, mang lại hiệu suất vượt trội so với các phương pháp truyền thống như SVM.

3.2.1 Tiền xử lý dữ liệu (Preprocessing)

Dữ liệu hình ảnh MNIST cần được chuẩn bị phù hợp cho mạng CNN. Quy trình xử lý gồm các bước sau:

- **Chuẩn hóa (Normalization):** Các giá trị pixel từ $[0, 255]$ được chuẩn hóa về $[0.0, 1.0]$ bằng công thức:

$$X_{norm} = \frac{X_{raw}}{255.0} \tag{3.5}$$

Điều này giúp ổn định quá trình huấn luyện và cải thiện hiệu suất.

- **Reshape dữ liệu:** Dữ liệu được định hình lại thành tensor 3 chiều $(28 \times 28 \times 1)$ để phù hợp với đầu vào của CNN:

$$X \in \mathbb{R}^{784} \xrightarrow{\text{Reshape}} X' \in \mathbb{R}^{28\times28\times1} \tag{3.6}$$

- **Mã hóa nhãn (One-hot encoding):** Nhãn được chuyển đổi từ số nguyên sang vector one-hot để phù hợp với hàm mất mát `categorical_crossentropy`.

3.2.2 Thiết kế kiến trúc mạng sequential

Bảng 3.1: Chi tiết cấu trúc các tầng trong mô hình CNN đề xuất

Tầng (Layer)	Tham số cấu hình	Chức năng
Conv2D_1	32 filters, size 3x3, ReLU	Trích xuất các đặc trưng cơ bản (cạnh, nét).
MaxPooling_1	Pool size 2x2	Giảm kích thước không gian, giữ đặc trưng mạnh nhất.
Conv2D_2	64 filters, size 3x3, ReLU	Trích xuất các đặc trưng phức tạp hơn.
MaxPooling_2	Pool size 2x2	Tiếp tục giảm chiều dữ liệu.
Conv2D_3	64 filters, size 3x3, ReLU	Trích xuất đặc trưng bậc cao.
Flatten	-	Chuyển đổi Tensor 3D thành Vector 1D.
Dense (Hidden)	64 units, ReLU	Kết hợp các đặc trưng để chuẩn bị phân loại.
Dense (Output)	10 units, Softmax	Trả về xác suất dự đoán cho 10 chữ số.

3.2.3 Triển khai (Implement)

a, Sơ đồ cấu trúc thư mục (Project Structure)

Dự án phân loại chữ số MNIST sử dụng CNN được tổ chức theo cấu trúc thư mục tương tự như mô hình SVM để đảm bảo tính nhất quán và dễ bảo trì:

- **main.py:** File chính để chạy chương trình, chứa hàm `run_cnn_project()` để thực thi mô hình CNN.
- **Data_preparation/:** Thư mục chứa các module xử lý dữ liệu.
  - `__init__.py`: File khởi tạo package.
  - `data_loader.py`: Chứa hàm `load_and_prepare_for_cnn()` để chuẩn bị dữ liệu cho CNN.
- **Models/:** Thư mục chứa các module định nghĩa mô hình.
  - `__init__.py`: File khởi tạo package.
  - `cnn_model.py`: Chứa các hàm xây dựng, huấn luyện, lưu và đánh giá

mô hình CNN.

- **Results/:** Thư mục lưu trữ kết quả và mô hình đã huấn luyện.
  - `mnist_cnn_model.h5`: Mô hình CNN đã được lưu.
  - `Result CNN.txt`: Kết quả đánh giá của mô hình CNN.

b, Quá trình huấn luyện AI

Quá trình huấn luyện mô hình CNN được thực hiện theo các bước chi tiết sau:

1. **Tải và tiền xử lý dữ liệu:**
   Sử dụng hàm `load_and_prepare_for_cnn()` từ module `Data_preparation/data_l`
   Dữ liệu MNIST được tải từ thư viện TensorFlow/Keras. Các bước tiền xử lý bao gồm:
   - Reshape dữ liệu từ (28, 28) thành (28, 28, 1) để phù hợp với đầu vào CNN (thêm chiều kênh màu).
   - Chuẩn hóa giá trị pixel từ khoảng [0, 255] về [0.0, 1.0] bằng cách chia cho 255.0.
   - Mã hóa nhãn thành dạng one-hot encoding sử dụng `to_categorical` để phù hợp với hàm mất mát categorical crossentropy.
2. **Xây dựng kiến trúc mạng:**
   Sử dụng hàm `build_cnn_model()` từ module `Models/cnn_model.py`.
   Mô hình bao gồm:
   - 3 lớp tích chập (Conv2D) với kích thước kernel 3×3 và hàm kích hoạt ReLU.
   - 2 lớp MaxPooling với pool size 2×2 để giảm chiều dữ liệu.
   - Lớp Flatten để chuyển đổi tensor thành vector.
   - 2 lớp Dense: một lớp ẩn với 64 units và ReLU, một lớp đầu ra với 10 units và Softmax.
   - Optimizer: Adam, Loss function: Categorical Crossentropy, Metrics: Accuracy.
3. **Huấn luyện mô hình:**
   Sử dụng hàm `train_cnn()` với các tham số:
   - Số epochs: 5 (đủ để hội tụ).
   - Batch size: 64.
   - Validation split: 10% dữ liệu huấn luyện được dùng để validation.

- Callback TimingCallback để theo dõi thời gian mỗi epoch.

4. **Lưu trữ mô hình:**
   Mô hình đã huấn luyện được lưu vào file `Results/mnist_cnn_model.h5` dưới định dạng HDF5 để có thể tái sử dụng.

c, Giải thích các hàm quan trọng (Code Snippets)

Trong phần triển khai CNN, các hàm quan trọng được định nghĩa trong file `Models/cnn_model.py`. Dưới đây là giải thích chi tiết:

Hàm `build_cnn_model(input_shape=(28, 28, 1), num_classes=10):`
Hàm này xây dựng kiến trúc mạng CNN sequential sử dụng Keras. Mạng bao gồm các lớp tích chập với MaxPooling để trích xuất đặc trưng, sau đó là các lớp fully connected để phân loại. Hàm compile mô hình với optimizer Adam và loss categorical crossentropy.

Hàm `train_cnn(model, X_train, y_train, epochs=10, batch_size=64)`
Hàm huấn luyện mô hình CNN với dữ liệu huấn luyện. Sử dụng validation split 10% và callback tùy chỉnh để theo dõi thời gian mỗi epoch. Trả về mô hình đã huấn luyện và lịch sử huấn luyện.

Hàm `save_cnn_model(model, filename='results/cnn_model.h5'):`
Lưu mô hình CNN đã huấn luyện dưới định dạng HDF5 sử dụng Keras.

Hàm `evaluate_cnn(model, X_test, y_test):` Đánh giá mô hình trên tập kiểm tra, tính độ chính xác, hiển thị ma trận nhầm lẫn và báo cáo phân loại. Sử dụng predict để chuyển đổi đầu ra softmax thành nhãn dự đoán.

Lớp `TimingCallback` là một callback tùy chỉnh để theo dõi thời gian huấn luyện mỗi epoch và hiển thị thông tin chi tiết.

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM

Chương này trình bày các kết quả đạt được sau quá trình huấn luyện và đánh giá hệ thống trên bộ dữ liệu MNIST. Các số liệu được thu thập dựa trên môi trường thực thi thực tế để đưa ra cái nhìn khách quan về hiệu năng của các thuật toán.

4.1 Kết quả thực nghiệm trên mô hình SVM

Mô hình SVM với cấu hình Kernel RBF và chiến lược One-vs-One (OvO) đã được huấn luyện trên toàn bộ tập dữ liệu 60,000 mẫu và đánh giá trên 10,000 mẫu kiểm tra.

4.1.1 Hiệu năng tổng quát

Các thông số về thời gian thực thi và độ chính xác tổng quát của mô hình được tóm tắt trong Bảng 4.1.

Bảng 4.1: Hiệu năng tổng quát của mô hình SVM

Thông số	**Giá trị thực nghiệm**
Thời gian huấn luyện (Training Time)	194.61 giây
Thời gian dự đoán trung bình	$\approx$ 0.02 s/ảnh
Độ chính xác cuối cùng (Final Accuracy)	**97.9200%**

Nhận xét: Với thời gian huấn luyện xấp xỉ 3 phút trên nền tảng phần cứng cá nhân, mô hình đã đạt được độ chính xác rất cao. Điều này minh chứng cho sức mạnh của Kernel RBF trong việc phân tách dữ liệu trong không gian đặc trưng 784 chiều của ảnh MNIST.

4.1.2 Phân tích ma trận nhầm lẫn (Confusion Matrix)

Ma trận nhầm lẫn giúp quan sát chi tiết khả năng phân loại của mô hình trên từng lớp chữ số, đồng thời xác định các cặp chữ số dễ gây nhầm lẫn nhất.

Bảng 4.2: Ma trận nhầm lẫn của mô hình SVM trên 10,000 mẫu kiểm tra

Thực tế \ Dự đoán	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0	973	0	1	0	0	2	1	1	2	0
1	0	1126	3	1	0	1	1	1	2	0
2	6	1	1006	2	1	0	2	7	6	1
3	0	0	2	995	0	2	0	5	5	1
4	0	0	5	0	961	0	3	0	2	11
5	2	0	0	9	0	871	4	1	4	1
6	6	2	0	0	2	3	944	0	1	0
7	0	6	11	1	1	0	0	996	2	11
8	3	0	2	6	3	2	2	3	950	3
9	3	4	1	7	10	2	1	7	4	970

Phân tích sai số: Dựa trên kết quả Bảng 4.2, mô hình gặp khó khăn nhất ở các cặp chữ số có sự tương đồng về cấu trúc hình học:

- **Cặp (4, 9):** Có 11 trường hợp chữ số 4 bị nhầm sang 9 và 10 trường hợp ngược lại. Sự nhầm lẫn này thường do nét viết tay của phần trên số 9 hoặc nét gạch ngang của số 4 không rõ ràng.
- **Cặp (2, 7):** Có 11 trường hợp chữ số 7 bị nhầm sang 2, cho thấy sự tương đồng ở nét gạch ngang và độ cong của thân chữ.

4.1.3 Chỉ số chi tiết (Classification Report)

Các chỉ số Precision, Recall và F1-score giúp đánh giá sự cân bằng của mô hình trên toàn bộ 10 lớp chữ số.

Bảng 4.3: Báo cáo kết quả chi tiết theo từng lớp (SVM)

Lớp (Digit)	Precision	Recall	F1-score	Số lượng mẫu
0	0.98	0.99	0.99	980
1	0.99	0.99	0.99	1135
2	0.98	0.97	0.98	1032
3	0.97	0.99	0.98	1010
4	0.98	0.98	0.98	982
5	0.99	0.98	0.98	892
6	0.99	0.99	0.99	958
7	0.98	0.97	0.97	1028
8	0.97	0.98	0.97	974
9	0.97	0.96	0.97	1009
Trung bình	**0.98**	**0.98**	**0.98**	**10,000**

Nhận xét: Chỉ số F1-score luôn duy trì ở mức từ 0.97 đến 0.99 cho tất cả các lớp. Điều này khẳng định chiến lược One-vs-One đã giúp mô hình không bị thiên lệch và đạt được sự ổn định rất cao trong việc phân loại đa lớp.

4.2 Kết quả thực nghiệm trên mô hình CNN

Mô hình CNN được huấn luyện thông qua 5 chu kỳ (epochs) với dữ liệu tensor 4D. Kết quả cho thấy tốc độ hội tụ nhanh và khả năng tổng quát hóa dữ liệu xuất sắc.

4.2.1 Hiệu năng tổng quát và Quá trình hội tụ

Hiệu năng của mạng CNN được ghi lại chi tiết qua từng epoch, cho thấy sự sụt giảm ổn định của hàm mất mát (loss) và sự tăng trưởng của độ chính xác (accuracy).

Bảng 4.4: Tóm tắt hiệu năng huấn luyện và kiểm tra của CNN

Thông số	**Giá trị thực nghiệm**
Số lượng Epochs	5
Thời gian huấn luyện tổng cộng	**47.16 giây**
Thời gian trung bình mỗi bước	11 ms/step
Độ chính xác trên tập kiểm tra (Accuracy)	**99.0500%**

Nhận xét: So với mô hình SVM, CNN có thời gian huấn luyện ngắn hơn đáng kể (47.16 giây so với 194.61 giây) nhờ vào khả năng tối ưu hóa song song của thư viện TensorFlow/Keras. Đồng thời, độ chính xác tăng từ 97.92% lên 99.05%, khẳng định ưu thế của việc trích xuất đặc trưng không gian qua các tầng tích chập.

4.2.2 Phân tích ma trận nhầm lẫn (Confusion Matrix)

Ma trận nhầm lẫn của CNN cho thấy các điểm dữ liệu bị phân loại sai đã giảm đi đáng kể so với SVM.

Bảng 4.5: Ma trận nhầm lẫn của mô hình CNN trên 10,000 mẫu kiểm tra

Thực tế \ Dự đoán	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0	977	0	1	0	0	0	0	1	1	0
1	0	1130	1	1	1	0	1	0	0	1
2	0	0	1029	1	1	0	0	1	0	0
3	0	0	4	1001	0	3	0	0	1	1
4	0	0	0	0	968	0	3	1	3	7
5	1	0	1	5	0	882	1	0	2	0
6	4	2	0	0	2	4	944	0	2	0
7	0	2	13	0	0	0	0	1007	1	5
8	2	0	1	2	0	0	0	0	968	1
9	0	0	0	0	3	3	0	1	3	999

Phân tích sai số: Mặc dù hiệu năng rất cao, mô hình vẫn còn một số ít nhầm lẫn:

- **Cặp (7, 2):** Có 13 trường hợp số 7 bị nhầm thành số 2. Đây vẫn là cặp số gây khó khăn nhất do sự tương đồng về cấu trúc nét xiên ngang.
- **Cặp (4, 9):** Số trường hợp nhầm lẫn đã giảm xuống chỉ còn 7 trường hợp (so với 11 trường hợp ở SVM), cho thấy các lớp tích chập đã trích xuất đặc trưng hình học tốt hơn.

4.2.3 Chỉ số chi tiết (Classification Report)

Bảng chỉ số chi tiết dưới đây minh chứng cho sự ổn định của mô hình trên toàn bộ 10 chữ số.

Bảng 4.6: Báo cáo kết quả chi tiết theo từng lớp (CNN)

Lớp (Digit)	Precision	Recall	F1-score	Số lượng mẫu
0	0.99	1.00	0.99	980
1	1.00	1.00	1.00	1135
2	0.98	1.00	0.99	1032
3	0.99	0.99	0.99	1010
4	0.99	0.99	0.99	982
5	0.99	0.99	0.99	892
6	0.99	0.99	0.99	958
7	1.00	0.98	0.99	1028
8	0.99	0.99	0.99	974
9	0.99	0.99	0.99	1009
Trung bình	**0.99**	**0.99**	**0.99**	**10,000**

Nhận xét: Với chỉ số F1-score đạt mức 0.99 cho hầu hết các lớp (riêng lớp số 1 đạt mức tuyệt đối 1.00), mô hình CNN đã thể hiện khả năng phân loại tốt, đặc biệt là sự cải thiện đáng kể ở lớp số 9 và số 2 so với mô hình SVM trước đó.

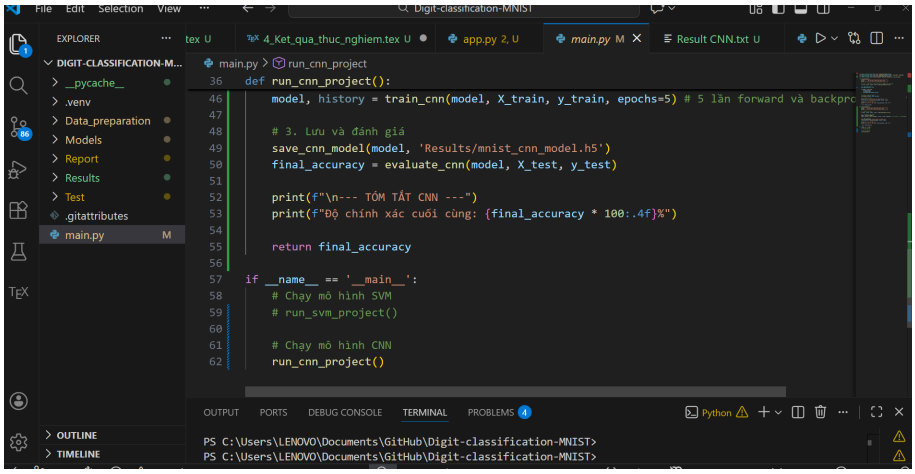
CHƯƠNG 5. DEMO

Chương này trình bày chi tiết quá trình thực nghiệm, bao gồm việc huấn luyện các mô hình SVM và CNN và kiểm tra hiệu suất của chúng trên tập dữ liệu MNIST.

5.1 Huấn luyện mô hình

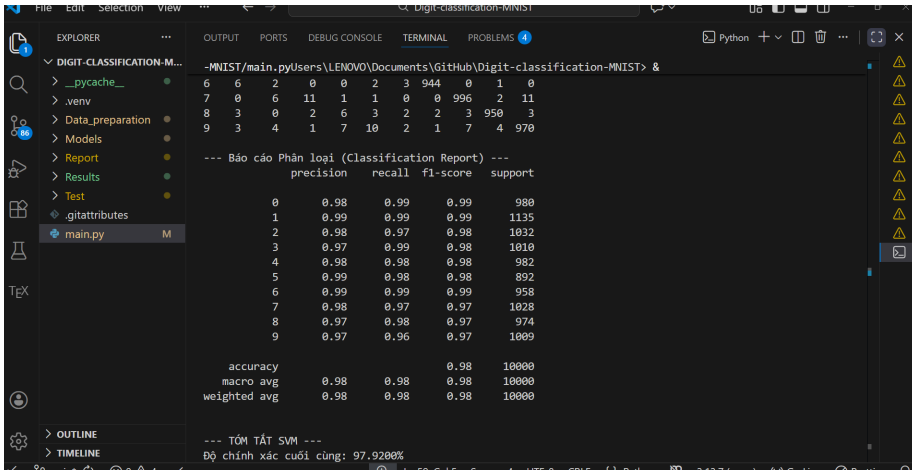
Để tiến hành huấn luyện mô hình SVM hoặc CNN, chúng ta có thể sử dụng các hàm tương ứng là `run_svm_project` hoặc `run_cnn_project` được định nghĩa trong file `main.py`. Kết quả của quá trình huấn luyện sẽ được hiển thị trực tiếp trên terminal, cung cấp thông tin về độ chính xác và các chỉ số đánh giá khác.

Hình 5.1 minh họa giao diện của file `main.py`, nơi người dùng có thể chọn và khởi chạy các hàm huấn luyện.

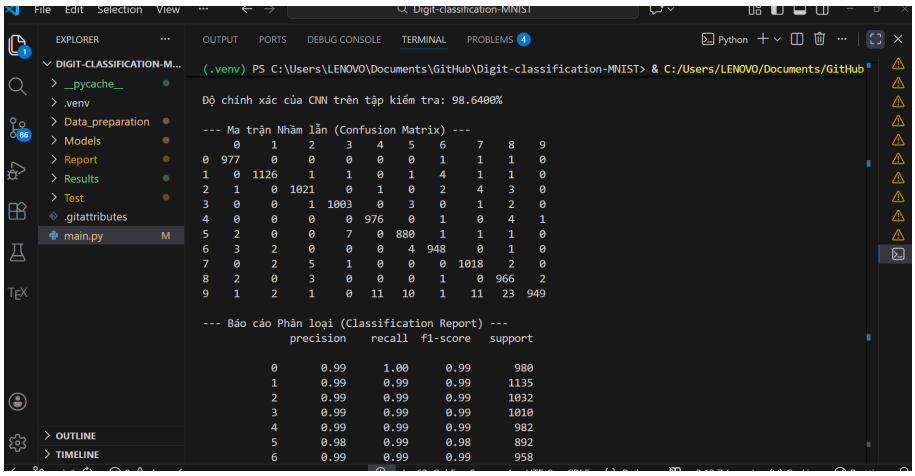


Hình 5.1: Giao diện file main.py

Hình 5.2 và Hình 5.3 lần lượt hiển thị kết quả huấn luyện của mô hình SVM và CNN trên terminal.



Hình 5.2: Kết quả huấn luyện mô hình SVM



Hình 5.3: Kết quả huấn luyện mô hình CNN

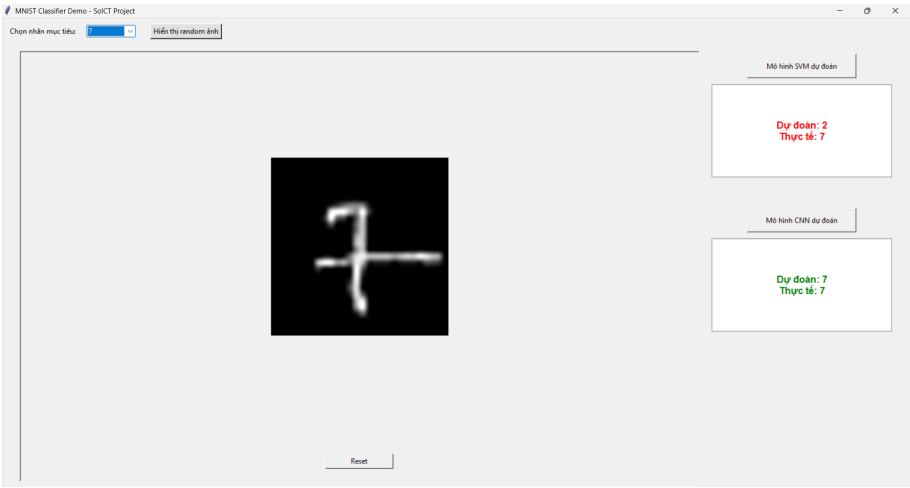
5.2 Kiểm tra mô hình

Để đánh giá hiệu suất của hai mô hình, em đã thiết kế một giao diện ứng dụng cho phép lấy ngẫu nhiên các ảnh chữ số dựa trên nhãn được chọn trước để tiến hành kiểm tra.

Hình 5.4 cho thấy cả hai mô hình AI đều nhận diện chính xác chữ số 3.

Hình 5.4: Cả hai mô hình nhận diện đúng chữ số 3

Trong khi đó, Hình 5.5 minh họa trường hợp mô hình SVM nhận diện sai chữ số 7 thành số 2, nhưng mô hình CNN vẫn nhận diện chính xác.



Hình 5.5: SVM nhận diện sai số 7 thành số 2, CNN nhận diện đúng

CHƯƠNG 6. KẾT LUẬN

Sau quá trình nghiên cứu, triển khai và thực nghiệm bài toán nhận diện chữ số viết tay trên bộ dữ liệu MNIST, báo cáo đã hoàn thành các mục tiêu đề ra với những kết quả cụ thể như sau:

- **Về mặt kỹ thuật:** Đã làm chủ và triển khai thành công hai phương pháp tiếp cận tiêu biểu: Học máy truyền thống (SVM với Kernel RBF) và Học sâu (CNN). Hệ thống đã được tổ chức theo mô hình mô-đun hóa chuyên nghiệp, từ khâu tiền xử lý dữ liệu đến huấn luyện và lưu trữ mô hình.
- **Về mặt thực nghiệm:**
  - Mô hình CNN cho thấy ưu thế vượt trội với độ chính xác đạt **99.05%** và thời gian huấn luyện tối ưu (47.16 giây).
  - Mô hình SVM đạt độ chính xác ổn định **97.92%**, khẳng định hiệu quả của chiến lược One-vs-One trong các bài toán phân loại đa lớp.
- **Về mặt ứng dụng:** Xây dựng thành công ứng dụng minh họa (Demo) với giao diện Tkinter, cho phép người dùng tương tác trực quan và kiểm chứng kết quả dự đoán của các mô hình trong thời gian thực.